Jenkins CI/CD Pipeline Project

Automated Build, Test and Deploy from GitHub using Jenkins and Docker

1. Objective

Automate the software delivery process so that every code push to GitHub triggers a Jenkins pipeline that builds the application, runs tests, creates a Docker image and deploys it to a server. This removes manual steps, catches errors early and speeds up releases.

2. Pipeline Workflow

Developer pushes code → GitHub webhook → Jenkins pipeline starts → Checkout → Build → Test → Docker image build → Push to Docker Hub → Deploy to server → Notification.

Tool	Role
Git / GitHub	Source code and webhook trigger
Jenkins	CI/CD automation server
Maven (or npm)	Build and unit tests
Docker / Docker Hub	Packaging and image registry
AWS EC2 (Ubuntu)	Jenkins server and deployment target

3. Prerequisites

• Two Ubuntu servers (or one for both): Jenkins server (t2.medium recommended) and an app server.

• A GitHub repository with a sample application and a Dockerfile.

• A Docker Hub account.

• Ports open: 8080 (Jenkins), 80 or app port (application), 22 (SSH from your IP).

4. Step-by-Step Implementation

Step 1: Install Java and Jenkins

```
sudo apt update
sudo apt install -y fontconfig openjdk-17-jre
sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/" | \
  sudo tee /etc/apt/sources.list.d/jenkins.list
sudo apt update && sudo apt install -y jenkins
sudo systemctl enable --now jenkins
```

Open **http://SERVER-IP:8080**, get the admin password with `sudo cat /var/lib/jenkins/secrets/initialAdminPassword`, install suggested plugins and create the admin user.

Step 2: Install Docker and grant access

```
sudo apt install -y docker.io
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Step 3: Install plugins and add credentials

• Plugins: Git, Pipeline, Docker Pipeline, GitHub Integration, SSH Agent.

• Manage Jenkins > Credentials: add **dockerhub-creds** (username/password) and **app-server-ssh** (SSH private key).

Step 4: Prepare the Dockerfile

```
FROM eclipse-temurin:17-jre
COPY target/app.jar /app/app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Step 5: Write the Jenkinsfile

Save this file as **Jenkinsfile** in the root of the repository.

```
pipeline {
  agent any
  environment {
    IMAGE = "yourdockerhub/myapp"
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build') {
      steps { sh 'mvn clean package -DskipTests' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Docker Build') {
      steps { sh 'docker build -t $IMAGE:$BUILD_NUMBER .' }
    }
    stage('Push Image') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub-creds',
            usernameVariable: 'U', passwordVariable: 'P')]) {
          sh 'echo $P | docker login -u $U --password-stdin'
          sh 'docker push $IMAGE:$BUILD_NUMBER'
        }
      }
    }
    stage('Deploy') {
      steps {
        sshagent(['app-server-ssh']) {
          sh '''ssh -o StrictHostKeyChecking=no ubuntu@APP_SERVER_IP "
            docker pull $IMAGE:$BUILD_NUMBER &&
            docker rm -f myapp || true &&
            docker run -d --name myapp -p 80:8080 $IMAGE:$BUILD_NUMBER"'''
        }
      }
    }
  }
  post {
    success { echo 'Deployment successful' }
    failure { echo 'Pipeline failed' }
  }
```

```
}
```

Step 6: Create the pipeline job

• New Item > Pipeline. Choose *Pipeline script from SCM*, SCM = Git.

• Enter the repository URL and branch, script path = Jenkinsfile.

• Tick *GitHub hook trigger for GITScm polling*.

Step 7: Add the GitHub webhook

• GitHub repo > Settings > Webhooks > Add webhook.

• Payload URL: **http://JENKINS-IP:8080/github-webhook/**, content type application/json, event: push.

5. Testing the Pipeline

• Commit a small change and push it to GitHub.

• Jenkins starts a build automatically; watch the Stage View turn green.

• Open http://APP-SERVER-IP and confirm the change is live.

• Break a unit test on purpose to confirm the pipeline stops at the Test stage.

6. Common Issues and Fixes

Problem	Fix
permission denied on docker.sock	Add jenkins to docker group, restart Jenkins
Webhook not triggering	Check port 8080 in security group and webhook URL
SSH deploy fails	Verify key credential and app server security group
Out of memory on build	Use a larger instance or add swap space

7. Best Practices

• Keep the Jenkinsfile in Git (pipeline as code).

• Store all secrets in Jenkins Credentials, never in code.

• Tag images with the build number, not only *latest*.

• Use HTTPS and restrict Jenkins access with roles.

8. Conclusion

The project delivers a working CI/CD flow: a push to GitHub automatically builds, tests, containerises and deploys the application. Future improvements include SonarQube code analysis, Kubernetes deployment, Slack notifications and multi-branch pipelines.